SoftwareX 34 (2026) 102621

Contents lists available at ScienceDirect

SoftwareX

journal homepage: www.elsevier.com/locate/softx

QSPLIT: A GUI-based testing framework for quantum split learning

Jae Hyun Cho [a], Min Jeon [a], Yae Jin Kwon [a], Kon-Woo Kwon [b], Youn Kyu Lee [a,*]

[a] Chung-Ang University, 84, Heukseok-ro, Dongjak-gu, Seoul, 06974, Republic of Korea
[b] Hongik University, 94, Wausan-ro, Mapo-gu, Seoul, 04066, Republic of Korea

ARTICLE INFO

Keywords:
Quantum neural networks
Split learning
Software testing framework
Code generation
Quantum computing

ABSTRACT

The implementation of Quantum Neural Networks (QNN) is challenging because it requires specialized knowledge of quantum mechanics, posing a significant obstacle for developers and limiting broader adoption. In this paper, we present QSPLIT, a GUI-based testing framework for efficient testing of QNN architectures within quantum split learning environments. QSPLIT allows developers to provide a subset of QNN components as target code for validation, while automatically generating the remaining components as dummy code to construct a complete architecture. The framework executes parallel split learning across multiple target–dummy code combinations, providing GUI-based tools for real-time log tracking, result comparison, and code export. By integrating these functionalities, QSPLIT streamlines the development and validation of QNN models. Its effectiveness was demonstrated through experiments on MedNIST, a medical imaging dataset.

Metadata

Code metadata (mandatory).

Nr.	Code metadata description	Code Metadata
C1	Current code version	v1.0.0
C2	Permanent link to code/repository used for this code version	https://github.com/JJowoo/QSPLIT
C3	Permanent link to Reproducible Capsule	N/A
C4	Legal Code License	GNU GPL v3.0
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Flutter, FastAPI, PyTorch, TorchQuantum, Python
C7	Compilation requirements, operating environments & dependencies	Flutter 3.32.8, FastAPI 0.115.12, PyTorch 2.8.0, TorchQuantum 0.1.8, Python 3.12.3
C8	If available Link to developer documentation/manual	See User Manual in GitHub repository
C9	Support email for questions	younkyul@cau.ac.kr

1. Motivation and significance

Recent advances in quantum computing have facilitated the development of Quantum Neural Networks (QNN), which implement neural networks on quantum hardware. A typical QNN architecture consists of three key components: State Encoder (SE), Parameterized Quantum Circuit (PQC), and Measurement (MEA). The SE converts classical data into quantum states by encoding input values into the phases and amplitudes of qubits. This is because quantum gates in a PQC operate on quantum states and cannot directly process classical vectors. Therefore, the SE is essential for transforming classical data into quantum states suitable for subsequent PQC processing. The PQC consists of quantum gates with trainable parameters that transform the encoded data using layers of single-qubit rotation gates and entangling gates. These gates

* Corresponding author.
Email address: younkyul@cau.ac.kr (Y.K. Lee).

https://doi.org/10.1016/j.softx.2026.102621
Received 19 September 2025; Received in revised form 11 March 2026; Accepted 19 March 2026
Available online 24 March 2026

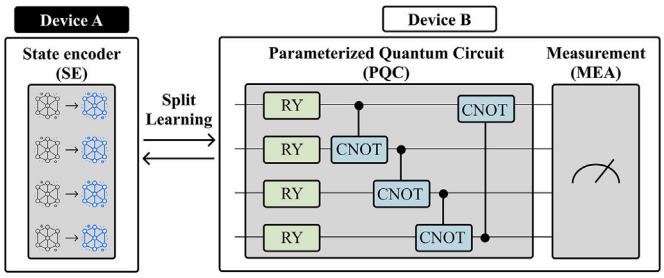

Fig. 1. An illustration of quantum split learning.

transform quantum states, and their parameters are optimized during the training process. Furthermore, the circuit architecture, defined by the types, number, and configuration of gates, directly impacts performance by influencing the model's expressiveness and training stability. The MEA stage measures the final quantum state and extracts expectation values or probability distributions for specified observables, which are used as classical outputs for downstream tasks.

This QNN architecture has recently been applied in various domains, including medicine and security [1,2]. Beyond these applications, recent studies have also explored methods to enhance sensitive information protection. As illustrated in Fig. 1, approaches including quantum split learning, which enable partial training at physically isolated nodes without sharing raw data, have been proposed [3]. In particular, quantum split learning significantly reduces the risk of exposing both data and model architecture compared to centralized QNN training, thereby enhancing data privacy in domains that handle sensitive information [4]. Furthermore, it mitigates computational overhead by distributing the model across multiple devices, with each device executing only a subset of the model [5]. Given these advantages, quantum split learning has been studied in high-privacy, high-trust domains such as healthcare and security [6].

However, the implementation of QNN inherently requires specialized knowledge of quantum mechanics, making their design and utilization particularly challenging for developers [7–9]. For example, to implement and evaluate a QNN using TorchQuantum framework [10], a developer is required to define the SE, construct the PQC as layers within the QuantumModule package, and convert measurement outcomes into classical data through the MEA. In this process, the encoding scheme, gate parameterization, and the shape of the measurement outputs need to be specified at the code level, which requires a solid understanding of quantum concepts and gate operations. Without such prerequisite knowledge, it is challenging to design and verify QNN code. Moreover, design choices such as circuit depth, entanglement structure, and encoding strategy directly affect the trainability of the model, and their impact is nontrivial to predict. For instance, depending on the circuit depth and gate layout in the PQC, the gradient variance can rapidly diminish due to the barren plateau phenomenon [11], potentially making QNN training infeasible. To address these challenges, several testing tools have been proposed to support the design and implementation of QNN and to provide automated code generation and verification [12–14].

EQuaTE [15] is a development tool that employs dynamic software testing techniques to analyze QNNs at the code level. It aims to improve training performance by tracking QNN training and mitigating the barren plateau phenomenon [11], which is analogous to local minima in conventional neural networks. AQUA [16] extends EQuaTE into a software validation framework for QNNs, supporting end-to-end testing from the State Encoder to the Measurement stage.

However, as shown in Table 1, existing tools do not support testing for a subset of QNN components, but instead require a complete QNN architecture. As a result, in situations where testing of an individual component is required, it becomes challenging to verify not only the validity of the component itself but also its compatibility with other components.

Table 1
Comparison of EQuaTE [15], AQUA [16], and QSPLIT.

Aspect	EQuaTE [15]	AQUA [16]	QSPLIT
Testing Objective	Trainability test for QNN architectures	Trainability and feasibility test for QNN architectures	Trainability and feasibility test for quantum split architectures
Test Target	Complete QNN architecture	Complete QNN architecture	Complete QNN architecture or individual components
Automated Code Generation Support	Not Supported	Not Supported	Supported
Parallel Execution Support	Not Supported	Not Supported	Supported
Split Learning Support	Not Supported	Not Supported	Supported
Output	Visualization of QNN training process	QNN architecture design checklist and feedback	Training results and executable dummy code

In split learning environments, developers may implement and test only a subset of components, as they typically lack both black-box and white-box access to the remaining components of the QNN architecture [17]. In such environments, each node typically contains only a subset of QNN components, thus limiting the feasibility of these tools [17–19]. Moreover, these tools do not support parallel execution for evaluating multiple QNN architectures concurrently, which increases the overall testing cost when testing must be repeated across different configurations. Here, parallel execution refers to the independent training of multiple target–dummy combinations simultaneously, without weight sharing or synchronization.

To address these limitations, this paper presents QSPLIT (Quantum SPlit Learning Integrated Tester), a GUI-based testing framework designed for efficient testing of quantum split learning architectures. In quantum split learning, each node contains only a subset of QNN components. Accordingly, QSPLIT allows developers to provide a specific component subset (i.e., target code) and automatically generates the remaining components (i.e., dummy code) to form various complete QNN architectures. By validating feasibility across multiple target–dummy code combinations, QSPLIT ensures that the target code demonstrates valid performance within diverse QNN configurations. Subsequently, QSPLIT performs split learning on the complete QNN architectures to assess both the performance and training efficiency of each combination. This process is executed in parallel across multiple architectures, facilitating efficient validation of the target code. QSPLIT provides the following key functionalities:

- **Automated Dummy Code Generation:** When developers provide only a subset of the QNN components, QSPLIT automatically generates dummy code for the remaining components to form a complete QNN architecture. This functionality enables testing even when constructing the complete QNN architecture is impractical, and it is also useful when specific components need to be validated individually. For example, if only the SE is provided among the core components (i.e., SE, PQC, and MEA), QSPLIT automatically generates compatible PQC and MEA to verify interoperability.
- **Parallel Split Learning and Log Tracking:** QSPLIT executes split learning in parallel by combining the target code with dummy code, enabling simultaneous testing of multiple combinations and thereby improving efficiency. During split learning, the training progress can be monitored in real-time through the GUI. This enables developers to assess whether the target code is functionally valid, compatible with dummy code, and stable in a quantum split learning environment. It also provides immediate feedback for code refinement and QNN architecture design.

- **Result Comparison and Code Export:** After split learning, developers can compare and analyze performance (e.g., accuracy) across different combinations and store the selected ones for reuse. This allows validated QNN components to be leveraged for follow-up development, thus improving efficiency.
- **GUI-Based Interaction:** All procedures, including component selection, hyperparameter configuration, split learning, and result comparison, are managed through the GUI. Each step is visually organized, providing an intuitive interface for training and evaluating the performance of QNN models within a quantum split learning environment.

These functionalities allow developers to compare training results, assess stability, and verify the feasibility of the target code. In particular, QSPLIT addresses the two challenges described above. First, even when a developer implements only a subset of the QNN components (i.e., SE, PQC, and MEA), QSPLIT automatically generates the remaining components and constructs a complete QNN architecture. This enables developers to validate the target code across various encoding schemes, quantum circuits, and measurement bases without the need to manually implement the missing components. Second, QSPLIT executes and compares multiple QNN configurations, helping developers identify trainable architectures. These capabilities, together with automatic code generation and parallelized split learning, accelerate testing across multiple configurations. They also facilitate the application of quantum split learning in sensitive domains such as medicine and security, which have been difficult to address with existing tools. In this paper, we demonstrate the effectiveness of the proposed framework through experiments on MedNIST, a medical imaging dataset [20]. The prototype of QSPLIT is available at: https://github.com/JJowoo/QSPLIT.git.

2. Software description

In this paper, we propose QSPLIT, a GUI-based software testing framework designed for quantum split learning testing. Given target code for any subset of QNN components, including State Encoder (SE), Parameterized Quantum Circuit (PQC), and Measurement (MEA), along with architectural hyperparameters (e.g., number of qubits, circuit depth), QSPLIT automatically generates dummy code for the remaining compatible components (e.g., PQC, MEA), constructs a complete QNN architecture, and executes split learning tests. In addition, it provides the functionality to compare and analyze results across different target–dummy code combinations, and to export selected dummy code.

2.1. Software architecture

In the current Noisy Intermediate-Scale Quantum (NISQ) era, iterative training that requires repeated circuit executions on quantum hardware faces practical challenges, including long queue latency, performance instability, and high operational costs [21–23]. As a result, many QNN studies, including those involving quantum split learning, have primarily been conducted in simulation environments [4,24]. QSPLIT is also designed with these limitations and operates within a quantum simulation environment (e.g., TorchQuantum), enabling flexible testing across diverse circuit configurations without relying on physical quantum hardware. This design allows for repeated experimentation and training without resource constraints. Moreover, quantum circuits developed within QSPLIT's simulation environment can be converted into IBM Qiskit's QuantumCircuit format, enabling execution on actual Quantum Processing Units (QPUs) [10]. This ensures that developers can easily transfer optimized models from the simulator to QPUs while preserving structural compatibility.

QSPLIT is implemented as a web-based GUI framework that supports distributed QNN testing. As summarized in the metadata, the frontend is implemented using Flutter, and the backend is implemented with the Python-based web framework FastAPI. For QNN modeling and code generation, QSPLIT adopts TorchQuantum, which was selected as the base package for its integration with PyTorch. This enables both a familiar development workflow and efficient batched execution for evaluating multiple target–dummy combinations [10,25–27]. While other libraries such as Qiskit and PennyLane also support quantum simulation and GPU acceleration, Qiskit was not adopted due to its limited native hardware support beyond IBM's quantum hardware system [25]. PennyLane was considered less suitable for the testing-centric workflow of this study, which involves repeated testing of many target–dummy combinations where execution speed is critical. TorchQuantum offers faster GPU-accelerated PQC training than PennyLane [10], which better aligns with this requirement. All quantum circuit simulation and training are conducted within the PyTorch framework in a unified Python environment. Data management is handled through temporary storage without a dedicated database, thereby enabling lightweight execution across diverse environments by eliminating the overhead associated with database maintenance. All data are managed dynamically on a per-experimental-session basis.

As illustrated in Fig. 2, QSPLIT consists of a frontend interface and backend modules. The frontend allows developers to upload target code, select components, configure hyperparameters, view results, and export

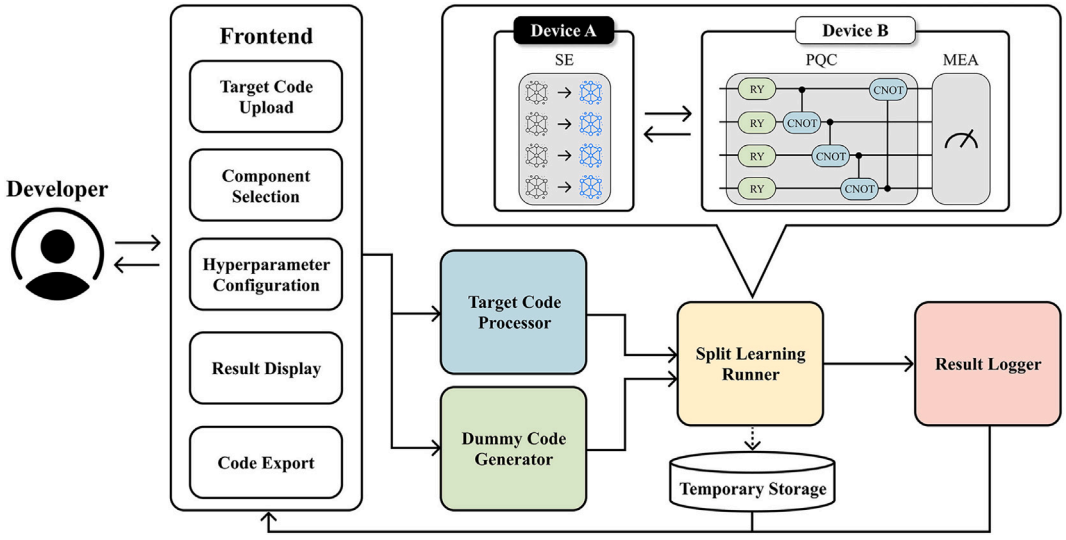

Fig. 2. Software architecture of QSPLIT.

code through the GUI. The backend performs core tasks through four primary modules: *Target Code Processor*, *Dummy Code Generator*, *Split Learning Runner*, and *Result Logger*.

All backend modules interact based on the developer-selected target code and automatically generated dummy code, for code generation and testing within a quantum split learning environment. Given the developer-provided target code and configuration, the *Target Code Processor* collects the selected components and provides them as target code to be paired with generated dummy code during split learning execution. The *Dummy Code Generator* then generates the missing components as dummy code and forms multiple combinations, each representing a complete QNN architecture. The *Split Learning Runner* receives the target code, dummy code, and training configuration from the previous modules, executes split learning for each combination in parallel, and produces training logs and performance metrics (i.e., loss, accuracy, and training time). During execution, the *Result Logger* transfers the training logs and performance metrics for each combination to the frontend in real-time. Therefore, developers can monitor training progress and performance across combinations through the GUI, verify that the target code is functionally valid in a quantum split learning environment, and compare the performance of different combinations.

2.2. Software functionalities

As illustrated in Fig. 3, QSPLIT offers a set of core functionalities for efficient testing of QNN architectures within a quantum split learning environment. As shown in Fig. 4, the developer workflow of QSPLIT proceeds as follows. First, the developer selects the component to be validated using the GUI. Subsequently, the developer uploads the target code corresponding to the selected component and configures the hyperparameters. Second, QSPLIT automatically generates dummy code based on the configured hyperparameters and predefined templates. The generated dummy codes are then combined with the target code to construct complete QNN architectures. Third, split learning is executed based on the constructed QNN architectures, during which the training progress is comprehensively recorded via log tracking. Finally, the training results are presented via the GUI, and the generated dummy code can be exported. The details and corresponding operating procedures are as follows:

- **Component Selection and Training Configuration:** As shown in Fig. 3(a), QSPLIT enables developers to select any subset of the three core QNN components (i.e., SE, PQC, and MEA) to be used as target code and to upload the corresponding implementation files. Based on this input, developers can specify the number of qubits, circuit depth, the dataset, the training hyperparameters, and the number of dummy codes to be generated. These settings are then applied for dummy code generation and split learning execution.
- **Automated Dummy Code Generation:** Developers can provide either a subset or all components of the QNN architecture as target code by clicking the 'Upload' button in the 'Part Selection' section. If all components are provided, QSPLIT directly performs testing on the complete QNN architecture. If only a subset is provided, QSPLIT automatically generates the remaining components as dummy code using predefined templates for SE, PQC, and MEA. For each missing component, QSPLIT instantiates a corresponding template and generates multiple dummy variants by sampling from a predefined pool of component configurations. These configurations are derived from the encoding schemes, gate compositions, and measurement bases supported by TorchQuantum [10], and are selected based on developer-specified settings such as the number of qubits, circuit depth, and training hyperparameters. For example, when generating a dummy PQC, QSPLIT selects the circuit depth and gate composition from template options and assigns qubit indices and entangling patterns consistent with the configured number of qubits. For a dummy MEA, it selects the measurement basis and assigns measured qubits to match the required output shape. In this process, QSPLIT ensures execution feasibility by maintaining structural compatibility between the generated dummy code and the provided target code. Once the hyperparameters are configured, developers can click the 'Generate' button in the 'Target Code Hyperparameter' section to generate dummy code. Each generated dummy is then paired with the target code to form a complete QNN architecture. As shown in Fig. 3(b), details of each dummy code are presented. For example, selecting Dummy#1 displays its

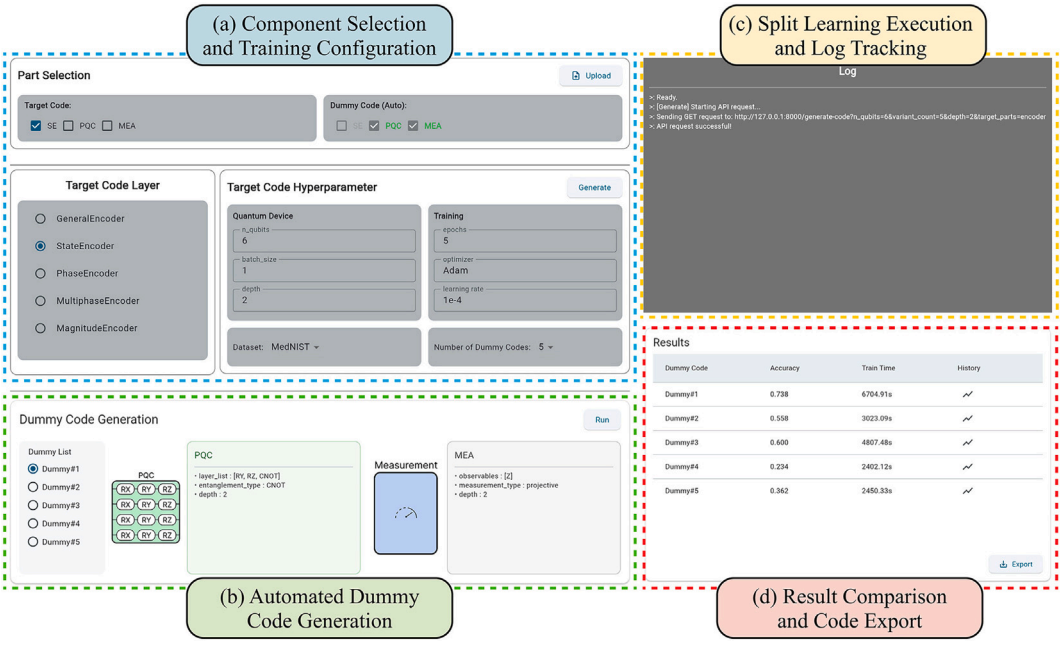

Fig. 3. A representative GUI showcasing the key functionalities provided by QSPLIT.

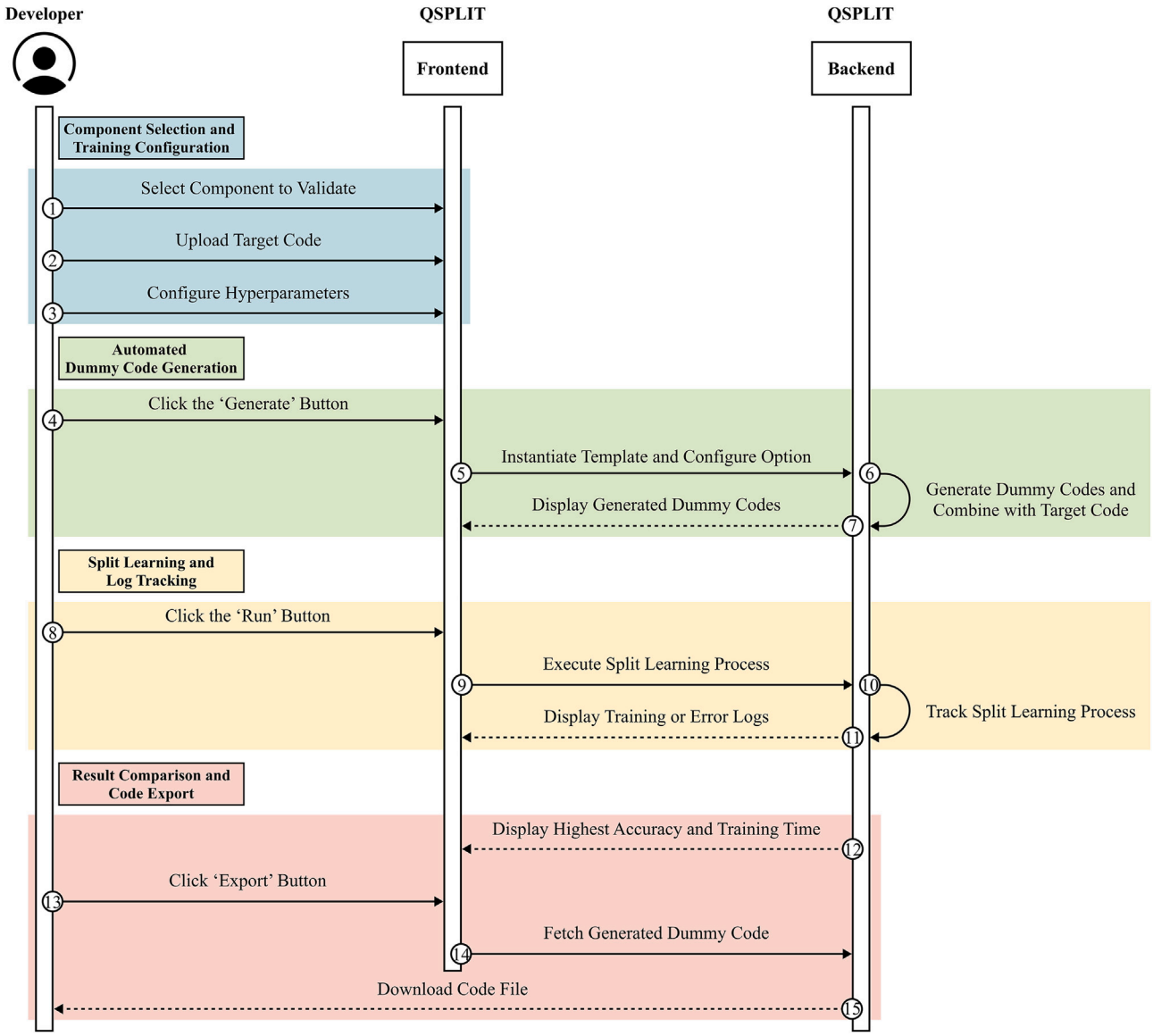

Fig. 4. Sequence diagram of QSPLIT.

PQC layer configuration and MEA measurement settings, combined with the uploaded target code. This enables developers to verify the composition and configuration of each dummy combination.

- **Split Learning Execution and Log Tracking:** By clicking the 'Run' button in the 'Dummy Code Generation' section, developers can initiate parallel split learning for different combinations. This process utilizes the TorchQuantum framework to perform training on the selected dataset. As shown in Fig. 3(c), the GUI displays training logs (i.e., training loss and accuracy) for each combination, thereby enabling developers to monitor training progress and assess trainability in real-time.
- **Result Comparison and Code Export:** As shown in Fig. 3(d), the GUI displays the testing results for each combination in list format, including the accuracy and the corresponding training time. Based on these results, developers can select a desired dummy code and click the 'Export' button in the 'Results' section. The selected dummy code is then exported as an executable Python file for a TorchQuantum-based environment. This allows the validated code to be reused for follow-up development, thereby improving development efficiency.

3. Illustrative examples

In this section, we present an example that demonstrates the quantum split learning testing workflow using the key functionalities of QSPLIT. This example targets MedNIST [20] dataset, which contains 58,954 2D medical images across six classes (i.e., AbdomenCT, BreastMRI, CXR, ChestCT, Hand, and HeadCT). We assume a quantum split learning scenario for a medical image classification task, where the developer implements only the SE, while the PQC and MEA components are inaccessible. This reflects privacy and security constraints in medical domains, where sensitive clinical images cannot be directly shared. This task aims to verify whether the provided SE can be combined with different dummy codes to form an executable and trainable QNN, and to identify effective combinations. The workflow involves selecting the SE as the target code, generating five dummy codes, executing split learning to compare training outcomes, and exporting the resulting code for reuse. The output includes the generated dummy code, training logs, the highest accuracy, and training time for each target–dummy code combination.

(a) Training progress of an effective target–dummy code combination.

(b) Training progress of an ineffective target–dummy code combination.

Fig. 5. Comparison of training curves for effective and ineffective target–dummy code combinations.

3.1. QNN component configuration

By clicking the 'Upload' button in the 'Part Selection' section, developers can provide either a subset or the entirety of a QNN architecture (i.e., SE, PQC, MEA) as the target code. Subsequently, the remaining architectural components are selected as dummy code. As shown in Fig. 3(a), the 'Part Selection' section enables developers to designate each component as either 'Target Code' or 'Dummy Code.' Because a component cannot be selected as both simultaneously, selecting SE as 'Target Code' automatically assigns the remaining components, PQC and MEA, as 'Dummy Code.' Developers can specify the detailed structure of the selected target code in the 'Target Code Layer' section. In this example, since the SE is chosen as the target code, the section displays a list of compatible encoders (e.g., GeneralEncoder, StateEncoder, PhaseEncoder, MultiphaseEncoder, and MagnitudeEncoder). The developer then selects 'StateEncoder' from this list. In the 'Target Code Hyperparameter' section, developers can configure the hyperparameters of the target code required for quantum split learning tests. In the 'Quantum Device' part, developers can configure hyperparameters for dummy code generation, such as the number of qubits, batch size, and circuit depth. The 'Training' part allows developers to configure hyperparameters for the training process itself, such as the number of epochs, the optimizer, and the learning rate.

The hyperparameters used in this example are as follows: number of qubits = 6, batch size = 1, circuit depth = 2, number of epochs = 30, optimizer = Adam, and learning rate = 1e-4. Rather than aiming to evaluate classification performance, this example is intended to demonstrate the QSPLIT workflow for verifying structural feasibility and trainability. The chosen settings (i.e., batch size and number of epochs) are selected for lightweight inspection and rapid testing, and are not intended to reflect ideal convergence conditions or optimized model performance. These hyperparameters were selected based on prior findings that many QNN models tend to converge rapidly and often reach near-optimal performance within five epochs [28]. For more rigorous testing, developers may configure a larger batch size and increase the number of training epochs. The MedNIST dataset was used for testing, selected from the datasets previously uploaded to QSPLIT.

3.2. Dummy code generation

Once the developer completes the configurations and clicks the 'Generate' button in the 'Target Code Hyperparameter' section, QSPLIT automatically generates dummy codes for the remaining components (i.e., PQC and MEA). The number of dummy codes is determined by the 'Number of Dummy Codes' hyperparameter, which is set to five in this example. The generated dummy codes follow a predefined template to ensure consistency in the input–output structure. Furthermore, these codes incorporate the specified hyperparameters to ensure architectural compatibility with the target code. As shown in Fig. 3(b), the details of the generated dummy codes are displayed in the 'Dummy Code Generation' section. This section allows developers to select a dummy code from the 'Dummy List' part to examine its internal structure. For instance, the PQC of Dummy#1 consists of RY, RZ, and CNOT gates, while its MEA consists of a Z observable. During execution, the target and dummy codes are deployed on separate devices, each serving as a functionally distinct execution unit for split learning.

3.3. Quantum split learning

By clicking the 'Run' button in the 'Dummy Code Generation' section, developers can initiate split learning. They can then click the graph icon in the 'Results' section to view the training curves for the selected target–dummy combination. Fig. 5(a) shows the training curves of a combination where training proceeds effectively, with the training loss consistently decreasing and the accuracy improving over epochs. In contrast, Fig. 5(b) shows the training curves of a combination where training fails to progress properly, as indicated by the absence of loss reduction and flat accuracy. This real-time comparison of training behavior across different combinations allows developers not only to verify the validity of the target code, but also to quickly identify which target–dummy code combinations are effective. Consequently, it facilitates the efficient exploration of combinations for configuring an optimal QNN architecture. If an error occurs, for example, a mismatch in the number of qubits due to an incorrect hyperparameter configuration or defects in the developer-uploaded target code during the part selection phase, split learning may fail. Listing 1 presents examples of the error diagnostic capabilities provided by QSPLIT. As shown in Listing 1(a), when the developer-provided target code contains syntax errors, QSPLIT reports error information via the GUI, including the exact file path and line number where the error occurred. This enables developers to identify and correct issues within the target code. As shown in Listing 1(b), execution errors arising from incorrect hyperparameter configuration may also occur. In this example, the developer sets the 'Number of Qubits' hyperparameter to 6, but the target code's actual number of qubits differs, causing a structural incompatibility with the dummy code. Based on the error information provided by QSPLIT, the developer can resolve the issue by following these steps: (1) examine the file path and line number reported in the logs; (2) compare the configured number of qubits with

```
[Dummy 1] COMPILE_ERROR in encoder at Backend/code/StateEncoder6Q.py:16
    >> tq.functional.rx(self.qdevice, wires=i, params=x[b][i] SyntaxError:
     '(' was never closed
[Dummy 2] COMPILE_ERROR in encoder at Backend/code/StateEncoder6Q.py:16
    >> tq.functional.rx(self.qdevice, wires=i, params=x[b][i] SyntaxError:
     '(' was never closed
```

(a) Error reporting for syntax errors in the developer-uploaded target code.

```
[Dummy 1] CONFIG_ERROR: encoder n_qubits mismatch (expected=6, provided=8)
[Dummy 2] CONFIG_ERROR: encoder n_qubits mismatch (expected=6, provided=8)
```

(b) Error reporting for a mismatch between hyperparameter configuration and the model structure.

Listing 1. Comparison of error reporting outputs for valid and invalid developer-uploaded target code.

Table 2
An example of accuracy and training time comparison provided by QSPLIT.

Dummy code	Accuracy	Training time
Dummy#1	0.738	6704.91 s
Dummy#2	0.558	3023.09 s
Dummy#3	0.600	4807.48 s
Dummy#4	0.234	2402.12 s
Dummy#5	0.362	2450.33 s

the actual number of output qubits; (3) adjust the hyperparameter or the target code to resolve the mismatch; and (4) regenerate dummy codes and re-execute the split learning process.

3.4. Result analysis and export

As shown in Table 2, once split learning is completed, the highest classification accuracy obtained during training (i.e., the maximum accuracy across epochs) and the corresponding training time for each combination are displayed. This allows developers to compare and analyze performance across various combinations. In this example, Dummy#1 achieved an accuracy of 0.738, while Dummies#2–#5 achieved 0.558, 0.600, 0.234, and 0.362, respectively. In addition, Dummy#1 required 6704.91 s for training, whereas Dummies#2–#5 required 3023.09, 4807.48, 2402.12, and 2450.33 s, respectively. Reporting both accuracy and training time enables developers to prioritize combinations by excluding underperforming or time-inefficient combinations. This also supports the selection of QNN architectures based on the accuracy–time trade-off. For example, Dummy#1 achieves the highest accuracy at the cost of substantially longer training time. Therefore, considering that accuracy is a critical requirement in medical image classification [29], the developer may select Dummy#1 as the final candidate despite its longer training time. After completing the analysis, the developer can export the desired dummy code as a set of Python files by clicking the 'Export' button in the 'Results' section. The exported files are organized into folders containing the dummy code files and an executable script. These files are compatible with the TorchQuantum framework, enabling developers to reproduce the validated code and its results. Because the dummy code follows the predefined code template's input–output format, it can be replaced or extended with other components sharing the same structure, thereby facilitating its use in subsequent development.

As illustrated in Fig. 6, each exported dummy folder includes its own executable script (i.e., *run.py*). For example, the developer can execute *run.py* for Dummy#1 to run the exported architecture, which consists of *MEA.py* and *PQC.py* in combination with the target code (i.e., *SE.py*), and inspect the training logs and evaluation outputs within their own environment, enabling rapid verification. In this example, Dummy#1 achieved a classification accuracy of 0.6983 on the MedNIST dataset, with a training time of 6801.23 s. Finally, after training, the resulting model can be used to classify the given dataset. To compare these results with other dummy codes (i.e., Dummy#2–#5), the developer can simply run the corresponding *run.py* in each exported folder and compare training metrics. Unlike conventional approaches that require manual implementation, such as writing QNN code, configuring training loops, and logging results for each configuration, QSPLIT automates the entire process, significantly reducing the setup time required for verification. Moreover, the exported code can serve as a baseline reference or be compared against manually developed QNN models, thereby supporting a seamless transition from architectural validation to practical deployment.

4. Impact

QSPLIT is a GUI-based testing framework designed to support developers in quantum split learning. While existing tools focus primarily on validating QNN architectures, QSPLIT extends this capability to the split learning environment. Through its GUI, QSPLIT offers functionalities including automated dummy code generation, parallel split learning execution, real-time log monitoring, result analysis for each target–dummy code combination, and code export. These functionalities mitigate the complexity of implementing quantum split learning, reducing the need

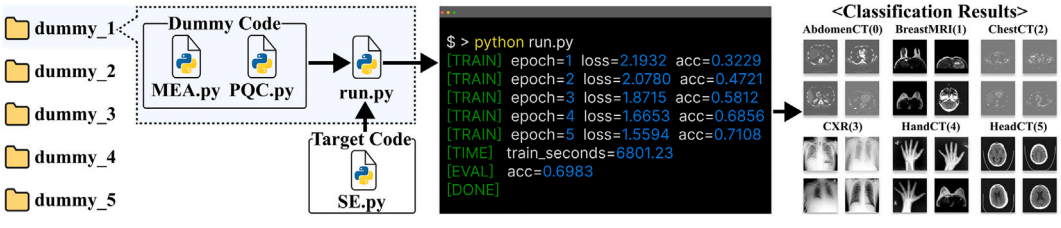

Fig. 6. Execution of the exported dummy code using *run.py* on the MedNIST dataset.

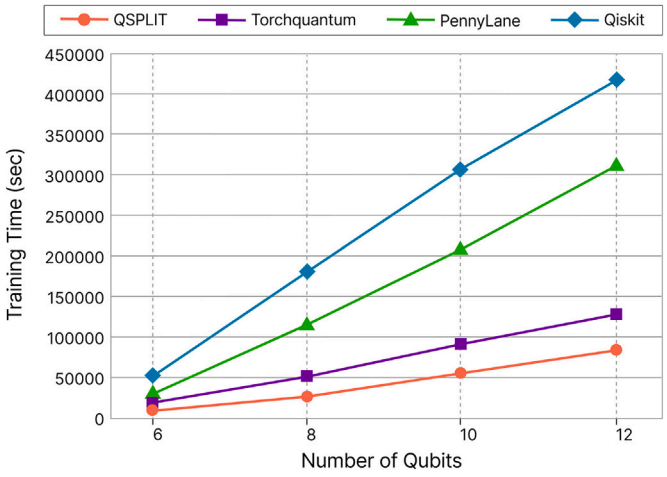

Fig. 7. Training time across different numbers of qubits comparing QSPLIT with existing sequential frameworks.

for extensive programming expertise and specialized knowledge, and thereby improving efficiency in both development and validation processes. To quantify the runtime benefit of parallel split learning provided by QSPLIT, we conducted a scalability evaluation by measuring training time while varying the number of qubits. The experiments were performed on a system equipped with an NVIDIA GeForce RTX 5080 GPU, AMD Ryzen 7800X3D CPU, and CUDA 12.0. As shown in Fig. 7, training time increases with the number of qubits, indicating a higher computational cost for larger circuits. Using the same model architecture and training configuration under identical hardware conditions, we compared QSPLIT with existing sequential execution frameworks, including TorchQuantum, Qiskit, and PennyLane. The results show that QSPLIT consistently achieves lower training times across all qubit settings. On average, QSPLIT reduces training time by 42.27% compared to TorchQuantum, and by 82.19% and 73.28% relative to Qiskit and PennyLane, respectively. This improvement is mainly attributable to QSPLIT's ability to evaluate multiple target–dummy code combinations concurrently, while the compared frameworks execute the same validation workflow sequentially in a local environment. These results demonstrate the practical efficiency gains achieved through QSPLIT's parallel execution when compared to conventional sequential execution approaches.

QSPLIT improves both the accessibility and usability of quantum split learning, enabling developers to efficiently validate QNN architectures in split learning environments. The GUI abstracts away the need for intricate programming, allowing developers to directly configure QNN architectures and execute split learning. By comparing results across various target–dummy code combinations, developers can intuitively assess the impact of QNN components' internal structures (e.g., the encoding method of the SE, the layer structure of the PQC, and the measurement strategy of the MEA) on overall QNN performance. Furthermore, even when developers provide only a subset of components, QSPLIT automatically generates the remaining components as dummy code and executes split learning in parallel. Consequently, QSPLIT provides a basis for validating both the reliability and performance of QNN architectures within quantum split learning environments in sensitive domains such as medicine and security, a capability not provided by existing tools.

QSPLIT also improves the efficiency of development workflows in quantum split learning. Developers are often burdened by time-consuming tasks such as implementing distributed execution code and configuring device-specific environments. QSPLIT mitigates this complexity by allowing developers to focus on hyperparameter optimization, performance evaluation, and architectural validation. In particular, its dummy code generation supports component-level verification, enabling rapid feasibility assessment of target code within a QNN architecture. Moreover, the generated dummy code can be exported as executable Python files, facilitating seamless integration into real-world applications.

5. Conclusions

In this paper, we propose QSPLIT, a GUI-based testing framework for quantum split learning designed to support developers. QSPLIT offers functionalities that mitigate the complexity of testing QNN architectures, thereby making the process more efficient and accessible. Consequently, QSPLIT streamlines the development and validation of QNN models, facilitating their application within quantum split learning environments. We have implemented QSPLIT as a web-based GUI framework and validated its usability and feasibility across diverse quantum split learning settings. QSPLIT's MedNIST example demonstrates that developers can validate target code in split learning environments by providing a subset of QNN components, generating the remaining components as dummy code, executing split learning across combinations, and monitoring real-time training logs. It also shows that QSPLIT facilitates error diagnosis and result analysis by reporting errors, accuracy, and training time for each combination. In addition, our scalability evaluation confirms that increasing the number of qubits leads to higher runtime costs. However, QSPLIT's parallel execution consistently reduces training time compared to sequential execution, achieving an average reduction of 42.26%. These results demonstrate the practical utility of QSPLIT in supporting verification workflows for quantum split learning.

In particular, QSPLIT enables early identification of QNN-specific challenges, such as component compatibility and trainability issues caused by increased circuit depth, prior to end-to-end execution during the architecture design phase. Moreover, since executing circuits on real quantum hardware incurs significant cost and latency, QSPLIT facilitates low-cost validation and iterative model refinement through repeated simulation-based testing across diverse configurations during the development phase.

A practical limitation of the current QSPLIT implementation is its reliance on predefined templates for dummy code generation. Consequently, supporting target code that contains circuit structures or measurement specifications beyond the provided templates (e.g., customized PQC gate compositions or non-default measurement bases) may require extension of the template set.

For future work, we plan to extend QSPLIT to support broader architectural diversity, including hybrid classical–quantum models, and to enable manual circuit design through more flexible templates. We also plan to support additional libraries, allowing the same testing workflow to run on frameworks such as Qiskit and PennyLane for extended execution flexibility. In addition, we aim to develop automated hyperparameter optimization methods, such as determining the number of qubits, to reduce reliance on manual trial-and-error configuration. Finally, we intend to extend QSPLIT to real quantum devices while considering the noise constraints of quantum hardware.

CRediT authorship contribution statement

Jae Hyun Cho: Writing – original draft, Software, Methodology, Conceptualization. **Min Jeon:** Writing – original draft, Validation, Software, Resources, Methodology, Data curation. **Yae Jin Kwon:** Writing – review & editing, Validation, Conceptualization. **Kon-Woo Kwon:** Writing – review & editing, Supervision. **Youn Kyu Lee:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by the Chung-Ang University Research Scholarship Grants in 2025. This work was also supported in part by the National Research Foundation of Korea (NRF) grants funded by the Korean government (MSIT) (No. RS-2025-16066331 and No. RS-2023-NR077140).

References

[1] Xiang Q, Li D, Hu Z, Yuan Y, Sun Y, Zhu Y, et al. Quantum classical hybrid convolutional neural networks for breast cancer diagnosis. Sci Rep 2024;14:24699.
[2] Kadry H, Farouk A, Zanaty EA, Reyad O. Intrusion detection model using optimized quantum neural network and elliptical curve cryptography for data security. Alex Eng J 2023;71:491–500.
[3] Araujo ARC, Okey OD, Saadi M, Adasme P, Rosa RL, Rodríguez DZ. Quantum-assisted federated intelligent diagnosis algorithm with variational training supported by 5G networks. Sci Rep 2024;14:26333.
[4] Cowlessur H, Thapa C, Alpcan T, Camtepe S. A hybrid quantum neural network for split learning. Quantum Machine Intelligence 2025;7:76.
[5] Kwak Y, Yun WJ, Kim JP, Cho H, Park J, Choi M, et al. Quantum distributed deep learning architectures: models, discussions, and applications. ICT Express 2023;9:486–91.
[6] Li Y, Qi Z, Li Y, Yang H, Shang R, Jiao L. Quantum splitting convolutional neural network-based distributed quantum disease detection model. Neurocomputing 2025:131410.
[7] Zeguendry A, Jarir Z, Quafafou M. Quantum machine learning: a review and case studies. Entropy 2023;25:287.
[8] Biamonte J, Wittek P, Pancotti N, Rebentrost P, Wiebe N, Lloyd S. Quantum machine learning. Nature 2017;549:195–202.
[9] Melnikov A, Kordzanganeh M, Alodjants A, Lee R-K. Quantum machine learning: from Physics to software engineering. Advances in Physics: X 2023;8:2165452.
[10] Wang H, Ding Y, Gu J, Lin Y, Pan DZ, Chong FT, et al. Quantumnas: noise-adaptive search for robust quantum circuits. In: Proceedings of the international symposium on High-Performance computer architecture; 2022. p. 692–708.
[11] McClean JR, Boixo S, Smelyanskiy VN, Babbush R, Neven H. Barren plateaus in quantum neural network training landscapes. Nature communications 2018;9:4812.
[12] Kwak Y, Yun WJ, Jung S, Kim J. Quantum neural networks: concepts, applications, and challenges. In: Proceedings of the International Conference on ubiquitous and future networks; 2021. p. 413–6.
[13] Zhao P, Wu X, Luo J, Li Z, Zhao J. An empirical study of bugs in quantum machine learning frameworks. In: Proceedings of the IEEE International Conference on quantum software; 2023. p. 68–75.
[14] Di Marcantonio F, Incudini M, Tezza D, Grossi M. Quantum advantage seeker with kernels (QuASK): a software framework to speed up the research in quantum machine learning. Quantum Machine Intelligence 2023;5:20.
[15] Park S, Feng H, Park C, Lee YK, Jung S, Kim J. EQuaTE: efficient quantum train engine for runtime dynamic analysis and visual feedback in autonomous driving. IEEE Internet Computing 2023;27:24–31.
[16] Park S, Baek H, Yoon JW, Lee YK, Kim J. AQUA: analytics-driven quantum neural network (QNN) user assistance for software validation. Future Gener Comput Syst 2024;159:545–56.
[17] Zhu X, Luo X, Wu Y, Jiang Y, Xiao X, Ooi BC. Passive inference attacks on split learning via adversarial regularization. arXiv arXiv:2310.10483 2023.
[18] Park S, Baek H, Kim J. Quantum split learning for privacy-preserving information management. In: Proceedings of the ACM International Conference on information and knowledge Management; 2023. p. 4239–43.
[19] Vepakomma P, Gupta O, Swedish T, Raskar R. Split learning for health: distributed deep learning without sharing raw patient data. arXiv arXiv:1812.00564 2018.
[20] apolanco3225. Medical MNIST classification. 2017. https://github.com/apolanco3225/Medical-MNIST-Classification.
[21] Ravi GS, Smith KN, Gokhale P, Chong FT. Quantum computing in the cloud: analyzing job and machine characteristics. In: Proceedings of the IEEE International Symposium on workload characterization; 2021. p. 39–50.
[22] Kurniawan H, Rodríguez-Soriano L, Cuomo D, Almudever CG, Herrero FG. On the use of calibration data in error-aware compilation techniques for NISQ devices. In: Proceedings of the IEEE International Conference on quantum Computing and Engineering, vol. 1. 2024. p. 338–48.
[23] Wu W, Wang Y, Yan G, Zhao Y, Zhang B, Yan J. On reducing the execution latency of superconducting quantum processors via quantum job scheduling. In: Proceedings of the IEEE/ACM International Conference on Computer-Aided design; 2024. p. 1–9.
[24] Kashif M, Marchisio A, Shafique M. Computational advantage in hybrid quantum neural networks: myth or reality? In: Proceedings of the ACM/IEEE Design Automation Conference; 2025. p. 1–7.
[25] Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, et al. Pytorch: an imperative style, high-performance deep learning library. Adv Neural Inf Process Syst 2019;32.
[26] Siddiqui S, Holzer J, Malcarne J, Wernsing GMB, Wyglinski AM. Framework for implementing quantum neural networks in wireless communications. IEEE Access 2025;13:91655–70.
[27] Zhan C, Gupta H. Quantum sensor network algorithms for transmitter localization. In: Proceedings of the IEEE International Conference on quantum Computing and Engineering, vol. 1. 2023. p. 659–69.
[28] Hickmann ML, Alves P, Quero D, Schwenker F, Rieser H-M. Hybrid quantum tensor networks for aeroelastic applications. Quantum Machine Intelligence 2025;7:103.
[29] Sushma SJ, Assegie TA, Vinutha DC, Padmashree S. An improved feature selection approach for chronic heart disease detection. Bull Electr Eng Inform 2021;10:3501–6.